

Web service deployment with a private CA

Follow-along: run the authority, issue the certificate, then break it three ways

Overview

Definition: Server certificate

A file the server sends a client at the start of a TLS session. It carries a public key, one or more names the holder of the matching private key is entitled to be identified by, a validity period with a start and an end, and a signature made by somebody else over all of that. The server proves it holds the matching private key as part of the handshake, so the certificate answers one question: given that this machine holds that key, which names may it claim, and who says so.

A web server in this lab already serves its site over TLS, and the client already refuses it. The handshake completes, the content is intact, and the **server certificate** the server presents was signed by nobody but the server itself. That is a working web service that no client will accept, and it is the state a great many internal deployments are left in. You fix it by building the piece that is missing: an authority that issues certificates, and a client that has been told to believe that authority.

The work is four things and they fail in four different places. An authority is a key that signs other keys' certificates. A certificate binds one name to one key for one span of time. A server sends not only its own certificate but every certificate needed to get from it up to the authority. And a client accepts none of it unless the authority at the top is one it was given in advance. Part 4 breaks three of those four on purpose, one at a time, because a learner who can reach a working deployment but cannot say which of the four a given failure came from has learned a recipe. Left out, because none of them changes any result here: certificate revocation, the cipher and key-exchange half of the handshake, ACME and the publicly trusted authorities a browser ships, and client certificates.

Running this lab in the TUI

You drive the lab from the MiniLabs launcher. Clone it if the machine does not have it, and pull if it does, because handouts and lab scripts change together:

```
git clone https://github.com/nzRichie/MiniLab.git ~/MiniLab # first time
cd ~/MiniLab && git pull # already cloned
```

Start the launcher from that directory:

```
cd ~/MiniLab && ./minilabs
```

If it exits with a Docker error, follow the setup in the repository's `README.md` and run it again.

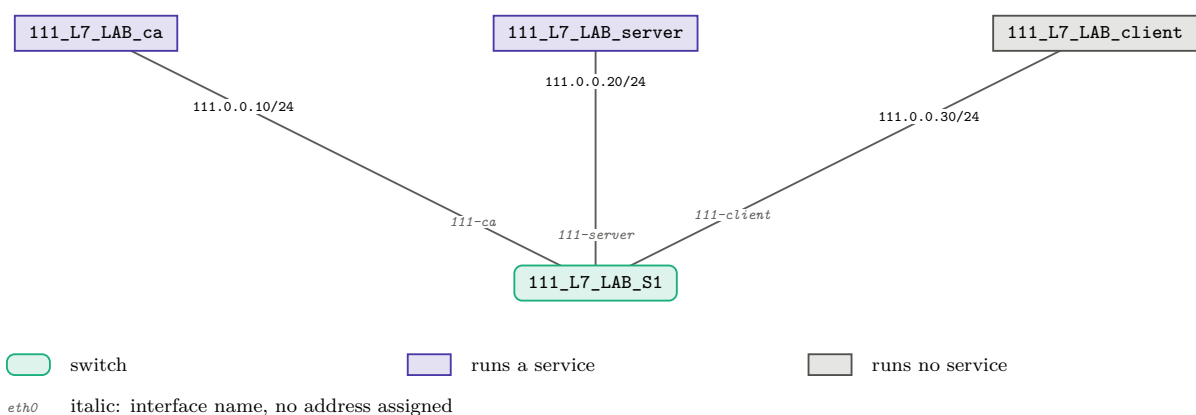
The root menu lists one category per row. Select **Host and service management**, then **Web service deployment with a private CA**, to reach this lab's actions:

- **Spawn** creates this lab's containers and wiring. Wait for the panel to turn green before you go on. The first spawn of a lab also builds its image; later spawns take seconds.

- **Status** prints the lab's state and its success oracle, so you read the before and after without shelling in. It also shows what the client made of the certificate the server last presented. Status takes over the whole screen: the arrow keys scroll its output, and Enter on **Done** closes it.
- **Shell command for a node** lists every node (ca, server, client) and the `docker exec -it ...` line that opens a shell on it. Copy the line for the node a step below names into your own terminal. It holds the screen until Enter on **Done**, as Status does.
- **Reset to baseline** throws away every certificate you issued, puts the web server back to the certificate it was delivered with, and empties the client's trust store, without tearing the lab down.
- **Teardown** removes the lab.
- **Open handout** opens this document.

Spawn the lab now, then work through Part 1 in your own terminal. **Back** (or Esc) returns to the lab list.

The setup



Four containers on one subnet, with one switch between them. Nothing routes and nothing filters: every host is one hop from every other, so nothing in the network can be blamed for a failure, and every failure you see comes from the certificate or from what the client believes about it.

Every container is named `111_L7_LAB_` followed by the name in the first column, so the web server is `111_L7_LAB_server`.

Device	Interface	Address	Runs
ca	111-S1	111.0.0.10	the certificate authority, and <code>sshd</code>
server	111-S1	111.0.0.20	<code>lighttpd</code> on <code>tcp/443</code>
client	111-S1	111.0.0.30	<code>openssl s_client</code> and <code>curl</code>
S1	111-ca, 111-server, 111-client	none	one Open vSwitch bridge

The client asks for the name `www.minilabs.lab`, not for an address, because a name is what a certificate carries and what a client checks the certificate against. That name resolves through an `/etc/hosts` entry the client was delivered with; there is no DNS server in this lab and name resolution is not what it teaches. A second name, `store.minilabs.lab`, resolves nowhere and exists only so Part 4 can issue a certificate for a name the client will not be asking for.

The certificate authority runs on its own machine and accepts SSH sessions and nothing else. The web server and the client each hold a key that machine accepts, so handing it a request or collecting a certificate is one `scp`. That arrangement is not what the lab grades; it is there so the two transfers in Part 3 are one command each.

Four commands do almost all of the work:

- `openssl req`: generate a key pair, and write either a self-signed certificate or a request for somebody else to sign.
- `openssl ca`: sign a request, record what was signed, and advance a counter.
- `openssl x509`: read one certificate and print any part of it.
- `openssl s_client`: open one TLS session, print the certificates that arrived, and print a number saying what verification made of them.

Open a shell wherever a step says so with:

```
# any of the three hosts: this is how every step below opens a shell
docker exec -it <container> bash
```

Assessment. This lab is graded on seventeen questions, marked inline below as [Question N](#), each placed at the step whose output it asks about. Answer all seventeen in a separate document as you work through the steps. Every question is answered from what you see on screen at that step, or from a prediction you make before running the next command.

Part 1: what the client makes of the certificate the server ships with

Before building anything, find out what the delivered deployment looks like from the one machine whose opinion matters.

Definition: Trust anchor, and the trust store

A certificate a client has decided in advance to believe, because somebody put it there rather than because anything vouched for it. The set of them on a machine is its trust store. Verification is the act of getting from the certificate a server presented to one certificate in that store, by following signatures; a certificate that no such path reaches is refused however well-formed it is. On this client the store is the file `/etc/ssl/certs/ca-certificates.crt`, and it starts out holding the public authorities Alpine ships and nothing else.

Definition: Verify return code

The number OpenSSL prints on the last line of an `s_client` session, saying what verification concluded. 0 is the only value meaning the certificate was accepted; every other value names one specific reason it was not, and the text in brackets beside it is the name of that reason. The number is worth recording rather than the text, because it is what a program can branch on: `X509_V_ERR_CERT_HAS_EXPIRED` is a constant with a fixed value, and the sentence beside it is not part of any interface.

1. **Open one TLS session and read what verification made of it.** Run this on the client, which holds no [trust anchor](#) from this lab.

```
# 111_L7_LAB_client: the machine that decides whether to believe the server
docker exec -it 111_L7_LAB_client bash
echo | openssl s_client -connect www.minilabs.lab:443 \
    -servername www.minilabs.lab -verify_hostname www.minilabs.lab
```

The output is long. The three lines that matter are near the top, where `s_client` prints the certificates that arrived as `s:` (subject, the name this certificate is about) and `i:` (issuer, the name that signed it), and a line near the end beginning `Verify return code:`, which carries the **verify return code**. That line is not the last one; a `--` separator follows it, which is why every later step in this lab greps for it by name rather than reading the tail of the output.

`-servername` sends the name in the TLS Server Name Indication extension, which is what a browser does and what makes this the same request `curl` will make. `-verify_hostname` is the one that is easy to leave off and expensive to leave off: `s_client` builds and checks the chain by default, and it checks no name at all unless it is given one, so without that option a certificate issued for a completely different host still reports code 0.

Question 1. Record the verify return code and the text beside it. Then record the `s:` and `i:` lines for the one certificate that arrived, and state which relationship between those two lines is what produced the code you got.

2. **Fetch the site twice, once with verification and once without.** `curl` verifies by default; `-k` turns that off and fetches anyway.

```
# 111_L7_LAB_client
curl https://www.minilabs.lab/ ; echo "exit $?"
curl -k https://www.minilabs.lab/ ; echo "exit $?"
```

Question 2. Record the exit status and the message from each of the two fetches. State what `-k` switched off, and say which of the two results establishes that the web service itself is working.

Definition: Self-signed certificate

One whose issuer and subject are the same name, signed with the private key belonging to the public key inside it. It is well-formed, it carries names and dates like any other, and the signature on it verifies. What it lacks is any statement by a second party: the only thing vouching for the key is the key. A root authority's own certificate is self-signed too, which is why being self-signed is not by itself a defect; what matters is whether the client was given that particular certificate in advance.

3. **Read the delivered certificate on the server.** The client saw it on the wire; this is the same file on disk.

```
# 111_L7_LAB_server: the machine serving the site
docker exec -it 111_L7_LAB_server bash
openssl x509 -in /etc/lighttpd/tls/selfsigned.crt -noout -subject -issuer
    -dates
openssl x509 -in /etc/lighttpd/tls/selfsigned.crt -noout -ext subjectAltName
```

It carries the name the client asked for and it is inside its dates. Every property a certificate can have on its own, this **self-signed certificate** has.

Question 3. This exact certificate, unmodified, on this exact server, could be made acceptable to this client by one change made somewhere else. Name that change, name the machine it is made on, and state what else that client would then accept as a result.

Part 2: the authority

An authority is a private key whose signature a client has been told to believe, plus the records of what it has signed. Building one is generating that key, writing a certificate for it, and then using it once.

Definition: Certificate authority, and the chain

A holder of a private key that signs other parties' certificates. Clients are given the authority's own certificate in advance, so a certificate the authority signed can be checked against a key the client already has. Authorities are stacked: a *root* signs an *intermediate*, the intermediate signs the certificates handed to servers, and the server's certificate together with every certificate between it and the root is the *chain*. In this lab the chain is `www.minilabs.lab` signed by `MiniLabs Issuing CA` signed by `MiniLabs Root CA`, and only the last of those three is what the client is given.

Two tiers rather than one, because the root's key is the one thing in the design that cannot be replaced quietly: it is the file every client was given a copy of. Keeping it idle, signing one certificate and then nothing, means the key that signs day to day is one that can be replaced without touching a single client.

Definition: basicConstraints and pathlen

An X.509 extension saying whether the certificate's key may sign other certificates. `CA:TRUE` permits it and `CA:FALSE` forbids it, and marking the extension **critical** means a client that does not understand the extension must refuse the certificate rather than ignore it. `pathlen:n` caps how many further authorities may appear below this one. `pathlen:0` on the intermediate in this lab means it may issue a certificate to `www.minilabs.lab`, and may not issue one to a third authority that would then issue its own.

Definition: Serial number, and the CA database

Every certificate an authority issues carries a serial number, unique within that authority, which is how a certificate is referred to without quoting the whole file. `openssl ca` reads the next one from a `serial` file, writes the issued certificate, advances that file, and appends one line to `index.txt` recording the status, the expiry, the serial and the subject. The root's counter in this lab starts at 1000 and the intermediate's at 2000, so a serial says at a glance which of the two authorities issued a certificate.

4. **Look at what the authority has been given and what it has not.** The directory tree, the two empty databases, the two counters and the configuration file are already here. Every key and every certificate is missing, and you create all of them.

```
# 111_L7_LAB_ca: the certificate authority
docker exec -it 111_L7_LAB_ca bash
ls -R /root/ca
cat /root/ca/int/serial /root/ca/int/index.txt
```

/root/ca/openssl.cnf is the file both authorities are driven from. Read the two sections named [root_ca] and [int_ca]: each names which key signs, which certificate that key belongs to, and where its database and counter live. openssl ca reads exactly one of those sections per run.

5. Create the root: its key, and a certificate it signs for itself.

```
# 111_L7_LAB_ca
openssl req -x509 -new -newkey rsa:2048 -sha256 -noenc -days 3650 \
    -config /root/ca/openssl.cnf -extensions v3_root_ca \
    -subj "/CN=MiniLabs Root CA" \
    -keyout /root/ca/root/private/root.key \
    -out /root/ca/root/certs/root.crt
chmod 600 /root/ca/root/private/root.key
```

-x509 is what makes this a certificate rather than a request, and it is the option that makes the result self-signed. -noenc leaves the private key unencrypted on disk, which a real root key would not be; it is spelled -nodes in OpenSSL 1.x and both spellings work in 3.x. Key generation prints a line of dots and plus signs to standard error while it runs; that is progress, not output.

6. Create the intermediate, and have the root sign it. Two commands: the intermediate generates a key and a request, and the root issues against that request.

```
# 111_L7_LAB_ca
openssl req -new -newkey rsa:2048 -sha256 -noenc \
    -config /root/ca/openssl.cnf -subj "/CN=MiniLabs Issuing CA" \
    -keyout /root/ca/int/private/int.key \
    -out /root/ca/int/csr/int.csr
chmod 600 /root/ca/int/private/int.key

openssl ca -config /root/ca/openssl.cnf -name root_ca -batch -notext \
    -extensions v3_intermediate_ca -days 1825 \
    -in /root/ca/int/csr/int.csr \
    -out /root/ca/int/certs/int.crt
```

-name root_ca picks the root's section instead of the default; -extensions v3_intermediate_ca picks the extension set that the issued certificate will carry.

Question 4. Print the **serial number** the intermediate's certificate was issued with (openssl x509 -noout -serial), then print the root's serial file and its index.txt. Give all three values, and state what the difference between the first and the second records.

7. Read the two extensions that decide what each authority may do.

```
# 111_L7_LAB_ca
openssl x509 -in /root/ca/root/certs/root.crt -noout -text | grep -A2 "Basic
Constraints"
openssl x509 -in /root/ca/int/certs/int.crt -noout -text | grep -A2 "Basic
Constraints"
```

Question 5. The intermediate carries `pathlen:0` and the root does not. Apply that to two specific cases: say whether the intermediate could issue a certificate to a second web server called `shop.minilabs.lab`, and whether it could issue one to a third authority that would then issue certificates of its own. Justify each from the value of the extension.

8. **Check that the intermediate verifies against the root.** This is what a client does on the wire, done on two files instead.

```
# 111_L7_LAB_ca
openssl verify -CAfile /root/ca/root/certs/root.crt /root/ca/int/certs/int.crt
```

Question 6. Both private keys on this machine can now cause a certificate to be accepted by a client that trusts the root. State what recovering from the theft of the intermediate's key requires, and state what recovering from the theft of the root's key requires that the first does not. Name the machines each recovery touches.

Question 7. You have built an authority and changed nothing on the web server. Predict the verify return code the client will report now, before running anything, and state in one sentence why building an authority changed nothing the client sees.

Check the prediction:

```
# 111_L7_LAB_client
echo | openssl s_client -connect www.minilabs.lab:443 \
    -servername www.minilabs.lab -verify_hostname www.minilabs.lab 2>/dev/null
\
| grep "Verify return code" | tail -1
```

Part 3: issue the certificate and deploy it

The server needs a certificate carrying `www.minilabs.lab`, signed by the intermediate. It never sends its private key anywhere to get one.

Definition: Certificate signing request

A file a server generates and sends to an authority, carrying its public key, the subject name it is asking for, and a signature made with the matching private key. That signature proves the sender holds the private key without disclosing it, and the private key never leaves the machine that generated it. What comes back is a certificate over the same public key, so the two files pair up and the key was never in transit.

Definition: subjectAltName

The extension carrying the names a certificate is valid for, each tagged with its kind (DNS: for a hostname, IP: for an address). A client compares the name it asked for against this list and against nothing else; the subject's Common Name was used for this before RFC 2818 was obsoleted by RFC 6125, and OpenSSL's own hostname check reads `subjectAltName` when one is present. A certificate for two names carries both, as `DNS:www.minilabs.lab`, `DNS:shop.minilabs.lab`.

9. **Generate the server's key and its request.** Both on the server, and the key stays there.

```
# 111_L7_LAB_server
docker exec -it 111_L7_LAB_server bash
openssl req -new -newkey rsa:2048 -sha256 -noenc \
  -subj "/CN=www.minilabs.lab" \
  -keyout /etc/lighttpd/tls/server.key \
  -out /etc/lighttpd/tls/server.csr
chmod 600 /etc/lighttpd/tls/server.key
```

Question 8. Print the **certificate signing request's** subject with `openssl req -noout -subject`, and print its full contents with `openssl req -noout -text`. State whether the request carries a `subjectAltName` of its own. You will issue the certificate in two steps' time; say now where the name in it is going to come from.

10. **Send the request to the authority.**

```
# 111_L7_LAB_server
scp /etc/lighttpd/tls/server.csr ca:/root/ca/int/csr/server.csr
```

11. **Write the extension set the certificate will carry, and issue it.** The authority's configuration sets `copy_extensions = none`, so nothing the request asks for reaches the certificate. Whatever the certificate carries, the authority put there, and this is where you put it.

```
# 111_L7_LAB_ca
cat > /root/ca/leaf.ext <<'EOF'
[ server_cert ]
basicConstraints          = CA:FALSE
keyUsage                  = critical, digitalSignature, keyEncipherment
extendedKeyUsage          = serverAuth
subjectKeyIdentifier      = hash
authorityKeyIdentifier    = keyid,issuer
subjectAltName            = DNS:www.minilabs.lab
EOF

openssl ca -config /root/ca/openssl.cnf -batch -notext \
  -extensions server_cert -extfile /root/ca/leaf.ext -days 825 \
  -in /root/ca/int/csr/server.csr \
  -out /root/ca/int/certs/server.crt
```

`CA:FALSE` means this key may not sign certificates, and `extendedKeyUsage = serverAuth` means it is for identifying a server rather than, say, signing code. The **subject alternative**

name line is the one the client will compare against what it asked for.

12. **Collect the certificate and the intermediate's certificate.** Two files, because the server has to send both.

```
# 111_L7_LAB_server
scp ca:/root/ca/int/certs/server.crt /etc/lighttpd/tls/server.crt
scp ca:/root/ca/int/certs/int.crt /etc/lighttpd/tls/int.crt
openssl x509 -in /etc/lighttpd/tls/server.crt -noout -subject -issuer -serial
openssl x509 -in /etc/lighttpd/tls/server.crt -noout -ext subjectAltName
```

Question 9. Print the public key held in the request and the one held in the issued certificate, with `openssl req -in ... -noout -pubkey` and `openssl x509 -in ... -noout -pubkey`, and say whether they are the same. Name the one file in `/etc/lighttpd/tls/` that must never have crossed to the authority, and state what in the request made sending it unnecessary.

Definition: PEM, and the order of a chain file

The text encoding these files use: the certificate's bytes in base64 between `---BEGIN CERTIFICATE---` and `---END CERTIFICATE---` lines. Because it is text, concatenating two certificates with `cat` makes a file holding both, which is how a chain is assembled. The order is fixed and it is not alphabetical or chronological: the server's own certificate first, then each certificate that signed the one before it. A client reads them in that order to work upward.

13. **Build the chain file and point the web server at it.** `ssl.pemfile` names the file lighttpd sends; `ssl.privkey` names the key that proves it holds the first certificate in that file.

```
# 111_L7_LAB_server
cat /etc/lighttpd/tls/server.crt /etc/lighttpd/tls/int.crt \
  > /etc/lighttpd/tls/chain.pem
grep -c "BEGIN CERTIFICATE" /etc/lighttpd/tls/chain.pem

sed -i 's|^(\ *ssl.pemfile *= *)\.|\1"/etc/lighttpd/tls/chain.pem"|' \
  /etc/lighttpd/lighttpd.conf
sed -i 's|^(\ *ssl.privkey *= *)\.|\1"/etc/lighttpd/tls/server.key"|' \
  /etc/lighttpd/lighttpd.conf
grep "ssl\." /etc/lighttpd/lighttpd.conf
```

Restart lighttpd so it reads the changed file. It removes its own pid file when it stops, so there is nothing to clean up between the two commands.

```
# 111_L7_LAB_server
pkill -x lighttpd
lighttpd -f /etc/lighttpd/lighttpd.conf
```

Question 10. The server now sends a complete **chain** and the client's trust store is still untouched. Predict the verify return code the client will report, and predict whether the number of certificates the client receives has changed since Part 1. Write both down, then run the block below and record what you got.

```
# 111_L7_LAB_client
echo | openssl s_client -connect www.minilabs.lab:443 \
    -servername www.minilabs.lab -verify_hostname www.minilabs.lab 2>/dev/null \
    | grep -E "^ *[0-9] s:|Verify return code"
curl https://www.minilabs.lab/ ; echo "exit $?"
```

14. **Give the client the root certificate.** It comes from the authority, over SSH, not from the server that is trying to be believed.

```
# 111_L7_LAB_client
docker exec -it 111_L7_LAB_client bash
scp ca:/root/ca/root/certs/root.crt \
    /usr/local/share/ca-certificates/minilabs-root.crt
update-ca-certificates
```

`update-ca-certificates` reads the certificates in `/usr/local/share/ca-certificates/` and rebuilds `/etc/ssl/certs/ca-certificates.crt`, the single file `curl` and `s_client -CAfile` both read. Copying into that bundle directly would work until the next time anything ran this command, which would rebuild the bundle from the directory and drop whatever was not in it.

Question 11. You installed the root and not the intermediate, and the server sends the intermediate itself. Installing the intermediate as the trust anchor instead would also make this fetch succeed. State one specific thing that arrangement would stop working for, and name the event that would break it.

15. **Fetch the site with verification on.**

```
# 111_L7_LAB_client
echo | openssl s_client -connect www.minilabs.lab:443 \
    -servername www.minilabs.lab -verify_hostname www.minilabs.lab 2>/dev/null \
    | grep "Verify return code" | tail -1
curl https://www.minilabs.lab/ ; echo "exit $?"
```

Question 12. Record the verify return code, `curl`'s exit status, and the marker string the page returned. Then state, for each of the four things Part 3 did (generated a key and a request, issued a certificate, assembled and served a chain, installed a trust anchor), which machine it changed.

Part 4: three ways it goes wrong

Everything above is now correct at once, which makes it a poor teacher: a deployment that works tells you nothing about which part was carrying the weight. You now break one property at a time and read what the client says. The three failures are chosen because each one is common, each one leaves the other three properties intact, and each one produces a different verify return code from the same server, the same client and the same authority.

Definition: Validity period

Two timestamps in every certificate: **notBefore**, from which it may be used, and **notAfter**, after which it may not. A client compares its own clock against both and refuses a certificate outside them, whatever else is correct about it. `openssl ca` takes the period either as `-days n` counted from now, or as an explicit `-startdate` and `-enddate` in the form `YYYYMMDDHHMMSSZ`, which is how a certificate can be issued already outside its own window.

16. Issue a certificate whose **validity period** has already ended, and serve it. The same request, the same authority, the same name; only the two dates differ.

```
# 111_L7_LAB_ca
openssl ca -config /root/ca/openssl.cnf -batch -notext \
  -extensions server_cert -extfile /root/ca/leaf.ext \
  -startdate 20240101000000Z -enddate 20240102000000Z \
  -in /root/ca/int/csr/server.csr \
  -out /root/ca/int/certs/expired.crt
```

```
# 111_L7_LAB_server
scp ca:/root/ca/int/certs/expired.crt /etc/lighttpd/tls/expired.crt
cat /etc/lighttpd/tls/expired.crt /etc/lighttpd/tls/int.crt \
  > /etc/lighttpd/tls/expired-chain.pem
sed -i 's|^(\ *ssl.pemfile *= *).*|\1"/etc/lighttpd/tls/expired-chain.pem"|' \
  /etc/lighttpd/lighttpd.conf
pkill -x lighttpd
lighttpd -f /etc/lighttpd/lighttpd.conf
```

lighttpd prints a line to standard error when it starts with a certificate outside its dates, and then serves it anyway. That is the message, and it is output rather than a command:

```
# 111_L7_LAB_server prints this; it is output, not commands to type
(..src/mod_openssl.c.2471) SSL: inactive/expired X509 certificate
```

```
# 111_L7_LAB_client
echo | openssl s_client -connect www.minilabs.lab:443 \
  -servername www.minilabs.lab -verify_hostname www.minilabs.lab 2>/dev/null \
  | grep "Verify return code" | tail -1
curl https://www.minilabs.lab/ ; echo "exit $?"
```

Question 13. Record the verify return code, its text, and `curl`'s exit status and message. Name the two fields of the certificate the client compared against its own clock, and name the two command-line options that set them in the issuing command above.

17. Issue a certificate for a name the client is not asking for, and serve it. Change one line of the extension file: `store.minilabs.lab` instead of `www.minilabs.lab`.

```
# 111_L7_LAB_ca
sed -i 's|^subjectAltName .*|subjectAltName          = DNS:store.minilabs.lab|' \
    /root/ca/leaf.ext
grep subjectAltName /root/ca/leaf.ext

openssl ca -config /root/ca/openssl.cnf -batch -notext \
    -extensions server_cert -extfile /root/ca/leaf.ext -days 825 \
    -in /root/ca/int/csr/server.csr \
    -out /root/ca/int/certs/wrongname.crt
```

```
# 111_L7_LAB_server
scp ca:/root/ca/int/certs/wrongname.crt /etc/lighttpd/tls/wrongname.crt
cat /etc/lighttpd/tls/wrongname.crt /etc/lighttpd/tls/int.crt \
    > /etc/lighttpd/tls/wrongname-chain.pem
sed -i 's|^(\( *ssl.pemfile *= *\) .*|\1"/etc/lighttpd/tls/wrongname-chain.pem"|' \
    /etc/lighttpd/lighttpd.conf
pkill -x lighttpd
lighttpd -f /etc/lighttpd/lighttpd.conf
```

```
# 111_L7_LAB_client
echo | openssl s_client -connect www.minilabs.lab:443 \
    -servername www.minilabs.lab -verify_hostname www.minilabs.lab 2>/dev/null \
    | grep "Verify return code" | tail -1
curl https://www.minilabs.lab/ ; echo "exit $?"
```

Question 14. Record the verify return code and `curl`'s message. This certificate was issued by an authority the client trusts, is inside its dates, and its signature verifies. Name the two strings the client compared to reject it, and say where each of the two came from: which command produced one, and which part of the client's own request produced the other.

18. **Serve the correct certificate without the intermediate.** No new certificate: point `lighttpd` at the leaf on its own.

```
# 111_L7_LAB_server
sed -i 's|^(\( *ssl.pemfile *= *\) .*|\1"/etc/lighttpd/tls/server.crt"|' \
    /etc/lighttpd/lighttpd.conf
pkill -x lighttpd
lighttpd -f /etc/lighttpd/lighttpd.conf
```

```
# 111_L7_LAB_client
echo | openssl s_client -connect www.minilabs.lab:443 \
    -servername www.minilabs.lab -verify_hostname www.minilabs.lab 2>/dev/null \
    | grep -cE "^ *[0-9] s:"
echo | openssl s_client -connect www.minilabs.lab:443 \
    -servername www.minilabs.lab -verify_hostname www.minilabs.lab 2>/dev/null \
    | grep "Verify return code" | tail -1
curl https://www.minilabs.lab/ ; echo "exit $?"
```

Question 15. Record how many certificates arrived, the verify return code, and `curl`'s message. Compare this code with the one you recorded in Question 10, before the trust anchor was installed. Both mean the client could not reach a trust anchor, so state precisely what was absent in each of the two cases, and say which side of the connection would have to change to fix each one.

Question 16. Across the four refusals you have now produced (Part 1, and the three in Part 4), `curl` reported the same exit status every time. Give that status. Two of the four also produced identical `curl` messages: name those two, and state what a monitoring system that reads only `curl`'s exit status and message would have to run instead to tell them apart.

19. Put the working deployment back.

```
# 111_L7_LAB_server
sed -i 's|^(\s*ssl.pemfile *= *)\s*.*|"/etc/lighttpd/tls/chain.pem"|' \
    /etc/lighttpd/lighttpd.conf
pkill -x lighttpd
lighttpd -f /etc/lighttpd/lighttpd.conf
```

20. Read what the authority recorded.

Every certificate it issued left a line behind, whether it was a good one or one of the two you broke on purpose.

```
# 111_L7_LAB_ca
cat /root/ca/int/index.txt
cat /root/ca/int/serial
```

Question 17. Give the number of rows in `index.txt` and the value in `serial`, and match each row to the certificate you issued that produced it. One row's expiry field is unlike the other two; say which row, and what about the way you issued it put that value there.

Checks: what “done” means

Run **Status** in the TUI, or `scripts/status.sh`. It reads the three machines rather than changing them, and it prints the authority, the certificate, what the server puts on the wire and what the client believes as four separate stages, so a red oracle can be traced to one of them. Done is every check ticked, which is these five things at once:

- The root certificate exists, carries `CA:TRUE`, and the intermediate verifies against it.

- The server's certificate was issued by `MiniLabs Issuing CA` and carries `DNS:www.minilabs.lab`.
- The server sends two certificates, not one.
- The client's trust store holds the root, and does not hold the intermediate.
- `openssl s_client` reports verify return code 0, and `curl` fetches the page with verification on and returns the marker.

The last line of the output says **TRUSTED** when all of them hold. Anything else prints the unticked checks; the first one is the earliest thing to fix, because every later stage is read through it.

If you want to start a part again from the delivered state, **Reset to baseline** throws away every certificate you issued, puts the server back to the certificate it signed for itself, empties the client's trust store of anything from this lab, and leaves the lab running.

Safety note

This lab is isolated. The four containers are wired to each other through one Open vSwitch bridge and nothing else; no container holds a route off this host, and the addresses in `111.0.0.0/8` exist only inside the lab. The authority you build is trusted by exactly one machine, the client this lab created, and its root certificate reaches no trust store anywhere else. The names `www.minilabs.lab` and `store.minilabs.lab` resolve only through the client's own `/etc/hosts`.

Tear the lab down

When you have finished, remove it. Pick **Teardown** in the TUI, or run `scripts/teardown.sh` directly. It removes this lab's four containers and nothing else.